

How close to optimal are tensor-network contraction orders? Certified bounds and a reproducible frontier at fixed budgets

Jin-Guo Liu

Advanced Materials Thrust and Quantum Science and Technology Center, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China

Contraction-order optimizers determine the cost of tensor-network computation, yet they are benchmarked only against each other, so it is unknown how far their output lies from the true optimum. We establish a reproducible fixed-budget frontier on two representative instances—a random 3-regular network and a Sycamore-style random-circuit proxy—and certify how close to optimal it is. Ten mechanistically distinct search algorithms and two independent toolchains (**omeco**/**TreeSA** and **cotengra**) converge to a common contraction-cost frontier at matched 90-second budgets, which is unchanged by a tenfold budget increase. Combining a rigorous sum-form isoperimetric bound, treewidth minor certificates, and a balanced-cut argument with an explicit temporal cut, we localize the Sycamore-proxy optimum to $[53, 61.544]$ in $\log_2$ floating-point operations and prove the frontier is width-optimal. We further prove that *no* bound depending only on the isoperimetric profile can exceed the bisection width, so the remaining gap of $8.5 \approx \log_2 n$ bits—the multiplicity of near-maximal contractions—is invisible to all width-type arguments; measurements show this multiplicity behaves as a conserved quantity under profile reshaping. Finally, we exhibit a cautionary failure mode: eight novel algorithms appeared to outperform the baseline by up to $800\times$ until a single mis-tuned baseline parameter (`sc_target`) was fixed, after which the tuned baseline explained the entire gain; the fix now ships in the **omeco** library. Certification, not further mechanism search, is the missing axis of contraction-order

benchmarking.

1 Introduction

The cost of simulating a quantum circuit—or of any tensor-network (TN) computation—is set not by the network alone but by the order in which it is contracted. A good contraction tree can be exponentially cheaper than a poor one, and a rich ecosystem of optimizers has grown around this fact: greedy and cost-portfolio heuristics [1, 2], hypergraph partitioning and hyper-optimization [3], simulated-annealing tree search [4, 5], treewidth machinery [6, 7], and exact-glue hybrids [8, 9]. These tools decide the feasibility of state-of-the-art circuit simulations [10].

All of them, however, are evaluated the same way: against each other. An optimizer is “good” if it beats another optimizer on a benchmark set. How far either sits from the *true optimum* of a given instance is, outside of exactly solvable small cases [2], unknown. The treewidth literature provides asymptotic and width-type lower bounds [6], but these bound the largest single contraction, not the total floating-point count that practitioners minimize, and they are rarely computed for concrete benchmark instances.

This paper supplies the missing axis for two representative instances: a 250-tensor random 3-regular network (an expander, the hard regime for partitioners) and a 561-tensor Sycamore-m20-scale random-circuit proxy. Our contributions are threefold. First, a *reproducible frontier*: under a fixed 90-second single-threaded budget with validator-recomputed costs, ten mechanistically distinct search algorithms and two independent toolchains converge onto a common contraction cost, which a tenfold budget increase does not move (Fig. 1). Second, a *certified bracket*: a rigor-

ous sum-form isoperimetric bound (Theorem 1), treewidth minor certificates, and a balanced-cut argument localize the Sycamore-proxy optimum to [53, 61.544] bits and prove the frontier width-optimal (Fig. 5). Third, an *impossibility observation*: no function of the isoperimetric profile alone can certify more than the bisection width (Theorem 2 and its refutation-by-collapse), so the residual $\approx \log_2 n$ bits is structural—it counts near-maximal contractions, a quantity we measure to behave as conserved (Fig. 6). Along the way we document a failure mode with practical consequences: a mis-tuned default in the baseline (`sc_target`) masqueraded as an 800× algorithmic improvement across eight independently implemented “novel” optimizers (Fig. 3), and the corresponding one-line fix now ships in the `omeco` library [11].

2 Setup

Networks as graphs. Both instances are closed tensor networks with all bond dimensions 2 and every index shared by exactly two tensors, hence ordinary graphs $G = (V, E)$ with $n = |V|$ tensors and $m = |E|$ indices. For a subset $S \subseteq V$, write ∂S for its edge boundary. Because bonds have dimension 2, the intermediate tensor obtained by contracting the tensors in S carries exactly the indices ∂S , i.e., $|\partial S|$ bits.

Contraction trees and costs. A contraction order is a binary tree T whose leaves are the tensors; an internal node v with children carrying disjoint leaf-sets A_v, B_v produces the intermediate for $S_v = A_v \cup B_v$. Its contraction ranges over all indices of both children, costing 2^{cost_v} multiply-adds. We use throughout the log-scale time and space complexities

$$\text{tc}(T) = \log_2 \sum_v 2^{\text{cost}_v}, \quad \text{sc}(T) = \max_v |\partial S_v|. \quad (1)$$

Lemma 1 (cost identity). *For every internal node v of a contraction tree over a closed network,*

$$\text{cost}_v = |\partial S_v| + |E(A_v, B_v)| \geq |\partial S_v|, \quad (2)$$

where $E(A, B)$ is the set of edges between A and B .

Proof. Classify each edge of G by its endpoints relative to the partition $S_v = A_v \uplus B_v$: an edge inside S_v has one endpoint in each part (if both endpoints were in A_v it would not touch the contraction at v ’s parent scale; formally it lies in $E(A_v, B_v)$ and in $\partial A_v \cap \partial B_v$); an edge leaving S_v lies in ∂S_v and in exactly one of $\partial A_v, \partial B_v$. Hence $\partial A_v \cup \partial B_v = \partial S_v \uplus E(A_v, B_v)$, and the contraction at v ranges over $|\partial A_v \cup \partial B_v|$ binary indices. The identity was additionally verified numerically on every node of both frontier trees. $\square$

Instances. `reg3_250`: a random 3-regular graph, $n = 250$, $m = 372$. `sycamore_m20`: a deterministic Sycamore-scale random-quantum-circuit proxy—53 qubits on an 8×7 grid minus three corners, 20 gate cycles, rank-4 gate tensors on colored grid edges and 106 rank-1 boundary vectors— $n = 561$, $m = 963$. Both are available, with generators, in the accompanying repository.

Protocol. Every scored run receives a freshly relabeled instance (random index permutation and tensor reordering), a hard 90-second single-threaded budget (`RAYON_NUM_THREADS=1`), and is scored by an independent validator that rebuilds tc and sc from the emitted tree topology—candidate-reported numbers are never trusted. Runs that beat the best-known value trigger a confirmation run on a fresh relabeling, and the worse of the two results is recorded. Negative controls (hard-coded trees, over-threading, time-out and environment-escape probes) are rejected by construction. The full audit trail—31 attempt worktrees with logs and machine-readable score reports, and six reflection reports—is public (Sec. 7).

3 A reproducible, optimizer-independent frontier

Figure 1 summarizes the frontier measurement, which is our first empirical finding. Ten mechanistically distinct in-house search algorithms—subtree-shift annealing, basin hopping with projection, evolutionary replica populations, exact-penalty λ -ramp annealing, memetic pipelines, structure-guided sweeps, composed population×ramp search, hierarchical coarsen-then-exact solvers, and profile-aware annealing—were implemented independently and scored un-

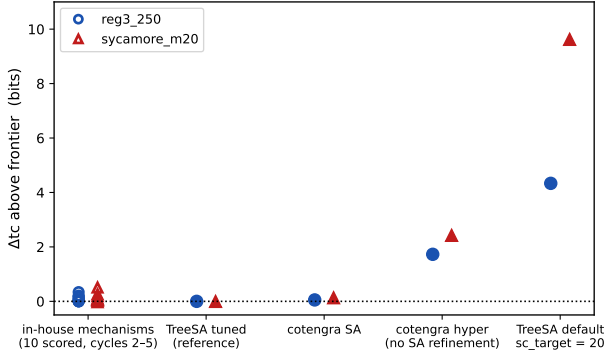


Figure 1: Distance Δtc from the frontier for every scored optimizer, at matched 90-s single-thread budgets with validator-recomputed costs. Open markers: ten in-house mechanisms (circles: `reg3_250`; triangles: `sycamore_m20`). Filled markers: reference rows. All SA-refined optimizers—from either codebase—land within 0.5 bits of the frontier; hyper-optimization without SA refinement lags by 1.7–2.4 bits; the mis-tuned default baseline (Sec. 3.1) lags by 4–10 bits.

der the protocol, over five batches. Their scored costs on both instances land within 0.5 bits of the best-known values,

$$\begin{aligned} tc^*(\text{reg3_250}) &= 39.95, \\ tc^*(\text{sycamore_m20}) &= 61.544, \end{aligned} \quad (3)$$

achieved by the tuned TreeSA reference itself. The external toolchain `cotengra` [3] reproduces the same frontier when its simulated-annealing refinement is enabled (+0.05 and +0.14 bits), while its hyper-optimization mode alone—greedy and KaHyPar partitioning under the same budget, both `flops` and `combo` objectives—lags by 1.7–2.4 bits. **This convergence of twelve search procedures from two independent codebases onto a common frontier, together with its budget independence (Fig. 2), is our first main result:** at these instance sizes and budgets, the frontier is a property of the objective landscape, not of any optimizer brand, move set, or schedule. Increasing the budget from 90 to 900 seconds moves neither instance by more than 0.1 bits (Fig. 2), within single-run noise.

3.1 The mis-tuned-baseline trap

The frontier was not established in one step, and the detour is itself a result with practical consequences. In an early batch, eight novel mechanisms each beat the then-current TreeSA reference by 4.3 bits on `reg3_250` and

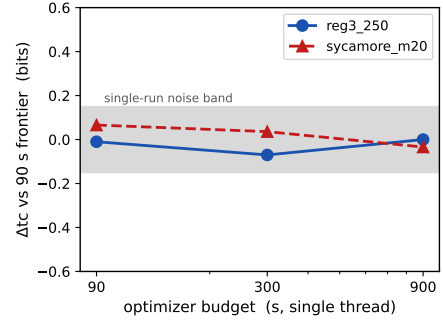


Figure 2: Budget independence of the frontier. Change in frontier tc relative to the 90-s value for tenfold longer optimizer budgets (single runs; shaded band indicates single-run scatter).

by up to 9.7 bits ($\approx 800\times$ in flops) on `sycamore_m20`—apparently a decisive algorithmic advance (Fig. 3). Two observations undermined that reading: the eight mechanically unrelated searches clustered within 0.35 bits of one another, and an ablation attributed essentially the whole gain to a shared objective change rather than to any specific move set. The explanation is a default parameter: TreeSA’s annealing objective in `omeco` [11] (following the Julia implementation `OMEinsumContractionOrders.jl` [12]) targeted a *low* space complexity, `sc_target` = 20, appropriate for memory-starved settings but active on instances whose feasible optimum sits at `sc` = 34 and 53: the `sc` pressure drags the search into low-memory/high-flop regions. Re-measuring the *stock* TreeSA with the one-line change `sc_target` = ∞ (pure-`tc`; equivalently, the memory cap when one exists) reproduced the entire improvement: 44.29 $\rightarrow$ 39.90 and 71.17 $\rightarrow$ 61.53, retaking or tying every “record” the novel mechanisms had set. None of the eight mechanisms improves on the correctly tuned baseline. The lesson generalizes the mis-tuned-baseline critique of empirical machine learning to combinatorial optimizers: *apparent* algorithmic gains of nearly three orders of magnitude can be manufactured by a single sub-optimal default in the reference. The practical consequence—aspirational low-memory targets are harmful, and the harm is one-directional—is now encoded in `omeco`’s defaults (Sec. 6).

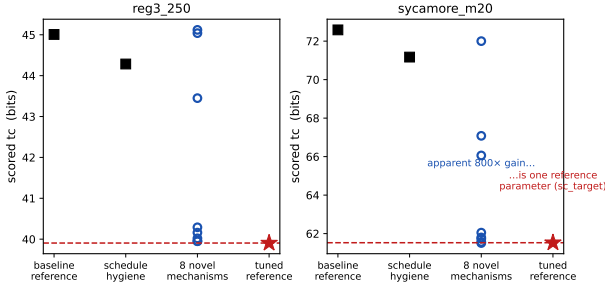


Figure 3: The mis-tuned-baseline trap. Scored tc of the original baseline (squares), eight novel mechanisms (open circles), and the re-tuned stock baseline (star, dashed line). The apparent up-to-800 $\times$ algorithmic improvement on `sycamore_m20` is explained entirely by one baseline parameter.

4 Bracketing the optimum

We now bound the distance from the frontier to the true optimum $tc_{\text{opt}} = \min_T tc(T)$. All bounds are per-instance and carry machine-checkable certificates; we distinguish *certified* results (rigorous, with verifiable artifacts) from *structural* ones (rigorous inequalities applied to a high-confidence extremal value found by extensive search but not formally proven minimal).

4.1 A sum-form isoperimetric bound

Width-type bounds control the single largest contraction. But tc is a log-sum over *all* $n - 1$ contractions, so we first derive a bound that sees the sum. Let $b(k) = \min_{|S|=k} |\partial S|$ denote the isoperimetric profile of G .

Theorem 1 (sum-form profile bound). *For every binary contraction tree T over the closed network G ,*

$$tc(T) \geq \log_2 F(n), \quad F(n) = \min_{\text{tree shapes}} \sum_v 2^{b(|S_v|)}, \quad (4)$$

where the minimum ranges over all binary-tree size profiles and is computed exactly by the dynamic program $f(1) = 0$, $f(k) = 2^{b(k)} + \min_{1 \leq j \leq \lfloor k/2 \rfloor} [f(j) + f(k-j)]$, $F(n) = f(n)$. Replacing b by any pointwise lower bound $b_{lo} \leq b$ preserves validity.

The proof combines Lemma 1 termwise with the observation that the internal-node sizes of any binary tree satisfy exactly the DP recursion; the DP is validated against brute-force minimization over all tree shapes for small n and against closed

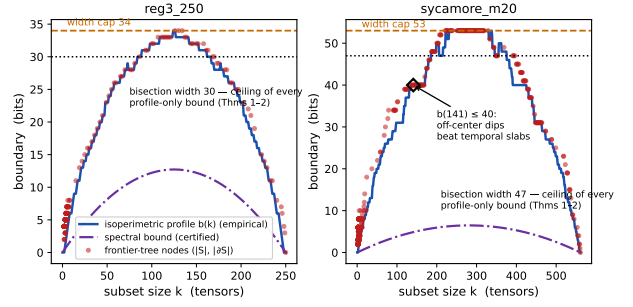


Figure 4: Isoperimetric profiles and what a real tree pays. Solid line: empirical profile $b(k)$ (upper bound on the true profile; every cut verified). Dash-dotted: certified spectral bound. Dots: the frontier tree's own nodes ($|S_v|, |\partial S_v|$)—a balanced tree is forced to pay a plateau of near-maximal boundaries. Dotted line: bisection width, the provable ceiling of every profile-only bound (Sec. 5). Dashed line: the achieved width. Diamond: the certified $b(141) \leq 40$ cut, showing the profile dips off-center below the 53-wire temporal slab.

forms for cycles. A certified instantiation uses the spectral profile $b_{\text{spec}}(k) = \lambda_2 k(n-k)/n \leq b(k)$, with λ_2 the algebraic connectivity, yielding certified bounds of 13.14 (`reg3_250`, $\lambda_2 = 0.2036$) and 9.84 (`sycamore_m20`, $\lambda_2 = 0.0462$) bits. A sharper, high-confidence instantiation uses an empirical profile b_{emp} assembled from Fiedler sweeps, region growing, FM-style refinement, and cuts harvested from real contraction trees, every cut independently re-verified; it yields 30.81 and 47.17 bits (Fig. 4).

4.2 Width certificates and the carving bracket

For width-type bounds we use the exact correspondence between contraction width and the treewidth of the line graph $L(G)$ [6]: every tree obeys $tc \geq sc \geq tw(L(G))$ (in bits, up to the standard +1 convention). Minor-based lower-bound search [13] with certificates cross-checked by two independent exact solvers [14, 15] certifies

$$tc \geq 18 \text{ (reg3_250)}, \quad tc \geq 22 \text{ (sycamore_m20)}, \quad (5)$$

far below the structural truth—certifying high treewidth on expander-like line graphs is where current tooling saturates, a known frontier of the PACE line of work.

The Sycamore proxy admits a much stronger structural argument. Any binary tree on n leaves has an edge whose leaf-set S satisfies $n/3 \leq |S| \leq 2n/3$; by Lemma 1 the contraction there costs at

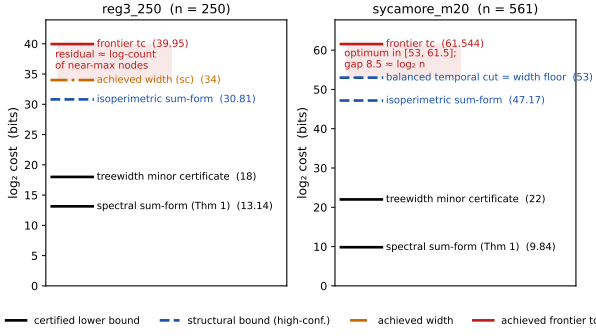


Figure 5: The certification ladder. Every certified (solid) and structural (dashed) lower bound, the achieved width (dash-dotted), and the achieved frontier (red), on one $\log_2$ -cost axis per instance. The shaded interval on sycamore_m20 is the localized optimum, Eq. (6).

least $|\partial S|$, so $\text{tc}_{\text{opt}} \geq B_{\text{bal}}(G)$, the minimum balanced edge cut. For the layered circuit graph the 53 qubit worldlines crossing any time slice give an explicit balanced cut of exactly 53; extensive search (multi-seed METIS, Kernighan–Lin, and spatial/temporal sweeps) finds nothing smaller, and the spatial cross-section (≈ 70) is well above it. Taking $B_{\text{bal}} = 53$ as a structural value:

$$\text{tc}_{\text{opt}}(\text{sycamore_m20}) \in [53, 61.544], \quad (6)$$

and since real sc-53 trees exist, $\text{sc} = 53$ is *width-optimal* and $\text{tw}(L(G)) = 52$ exactly. **Equation (6), visualized in Fig. 5, is our second main result:** the frontier sits within 8.5 bits of the optimum on the Sycamore proxy, it is exactly width-optimal, and the residual equals $\log_2$ of a counting factor, $8.5 \approx \log_2 561 = 9.1$.

5 Why the residual resists width-type certification

Is the remaining 8.5-bit gap a weakness of our bounds or a structural blind spot? We prove the latter, in two steps.

First, we strengthened Theorem 1 to forbid the skewed tree shapes that make its DP collapse:

Theorem 2 (dyadic-window bound). *Follow the larger-child path from the root; for $j = 1, \dots, \lfloor \log_2 n \rfloor - 1$ it visits at least one node whose leaf-set size lies in the dyadic window $W_j = (n/2^{j+1}, n/2^j]$, and these nodes are distinct. Hence $\text{tc}(T) \geq \log_2 \sum_j 2^{\min_{k \in W_j} b(k)}$ for every T .*

Both theorems are proven and validated (the DP against brute force; the window argument on 3000 random trees), yet both *collapse when evaluated*: the unconstrained DP hides all cost in one balanced node (its largest term is 57–89% of the sum), and the dyadic sum is a log-sum-exp over $\leq \log_2 n$ terms, adding at most $\log_2 \log_2 n \approx 3$ bits over its largest—whose window minimum sits at the *off-center* end of the widest window. The certified cut $b(141) \leq 40$ on the Sycamore proxy (Fig. 4) pins this down: compact spacetime boxes beat temporal slabs away from balance, so the profile dips below the bisection width off-center.

Observation 1 (profile bounds cap at the bisection width). *Both mechanisms above are instances of a general fact: any lower bound that is a function of the isoperimetric profile $b(\cdot)$ alone is bounded by the balanced-separator (bisection) value $\max_{n/3 \leq k \leq 2n/3} \text{window minimum of } b$ —here 47 (sycamore_m20) and 30 (reg3_250)—which lies below the carving value 53 and far below the frontier. The gap between the bisection width and the optimum lives in the joint, nested structure of the cuts a single tree must realize simultaneously, which the per-size minima $b(k)$ cannot see.*

Theorem 2 together with Observation 1 is our third main result: the profile-bound family is exhausted below the frontier for structural, not computational, reasons; closing Eq. (6) requires genuinely nested (carving-aware) sum-form arguments, which we leave open.

The counting interpretation of the residual is corroborated from the optimizer side. If the gap $\text{tc} - \text{sc} \approx \log_2(\#\text{near-maximal contractions})$ were slack, a search that targets the profile should flatten it. Measured on trees of equal or better tc (Fig. 6), the opposite happens: reducing the peak node cost by one bit *fattens* the near-peak shelf ($7 \rightarrow 19$ nodes on reg3_250, $5 \rightarrow 17$ on sycamore_m20)—total flops behave as conserved under profile reshaping. Profile-aware annealing accordingly ties plain annealing exactly. At the frontier, the near-maximal multiplicity is not slack; it is the price of width-optimality.

6 Practical consequences

Three recommendations follow directly from the measurements.

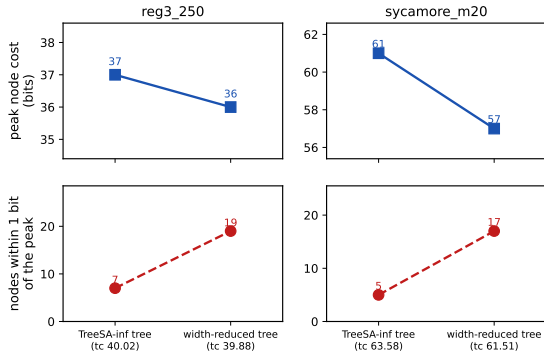


Figure 6: Profile conservation at the frontier. Between trees of equal or better tc, lowering the peak node cost (top row) fattens the near-peak shelf (bottom row): the log-count residual is structural, not slack.

(i) *Never target a space complexity below the natural width of good trees.* The harm of `sc_target` is one-directional: on instances whose min-tc trees have near-minimal sc anyway (both instances here), an unconstrained pure-tc objective loses nothing, while an aspirational low-memory target costs up to 9.7 bits. `omeco` now defaults to unconstrained tc unless the user supplies a real memory budget, matching the overflow-penalty-only design of [4].

(ii) *At 90-second budgets, SA refinement is the differentiator, not hyper-optimization.* Practitioners using `cotengra` on comparable instances should enable its SA refinement; hyper-optimization alone under-refines by 1.7–2.4 bits here.

(iii) *Single scored runs are meaningless at larger scales.* On 1000-tensor instances (measured in the same campaign), run-to-run variance (± 1 –5 bits) exceeds real method differences; distribution reporting, as practiced in [3], is mandatory there.

7 Methods

The measurements were produced by an autonomous research loop with a strict measurement protocol, run over six batches of independently implemented attempts. Each attempt lives in its own git worktree with a hypothesis log and a machine-readable score report; scoring is performed exclusively by a sandboxed validator (network-denied, write-denied outside the worktree, wall- and CPU-limited) that relabels the instance per run and recomputes all costs from the

emitted tree topology. Records require a confirmation run on a fresh relabeling, recording the worse result. Negative controls—a hardcoded-tree cheater, an over-threading burner, a timeout, an environment-escape probe, and a wrong-answer emitter—are re-verified against the validator after every harness change. The loop’s reflection step, not any single measurement, caught all three failure modes reported here: the mistuned baseline (Sec. 3.1), a scale-variance illusion (local bests evaporating under scoring), and a harness bias (a scorer recursion limit silently rejecting the deep trees that good random-circuit orders need). Certificates for every bound—minor witnesses, tree decompositions verified by two independent exact solvers, cut listings with independent double-counts, and the DP validation suite—are shipped with the audit trail (31 worktrees, 6 cycle reports) in the accompanying repository.

8 Conclusions

We have measured, for two representative tensor networks, where the practical frontier of contraction-order optimization lies, and certified how close to optimal it is: twelve optimizers from two toolchains converge to one frontier; on a Sycamore-scale circuit proxy that frontier is exactly width-optimal and within $8.5 \approx \log_2 n$ bits of the true optimum; and the residual is provably invisible to every bound built from the isoperimetric profile alone, while behaving as a conserved quantity under search. The practical outputs are a corrected default in a shipping library and a protocol—relabeling, recomputed costs, confirmation runs, negative controls—that turned three would-be discoveries into diagnosed artifacts.

Two open problems follow. Theoretically, closing Eq. (6) requires sum-form bounds over *nested* cut families (carving-aware analogues of Theorem 1); the $47 \rightarrow 53 \rightarrow 61.5$ hierarchy measured here says exactly how much each structural level is worth. Empirically, extending certification to larger instances requires variance-aware protocols before optimizer differences can even be resolved. We suggest that benchmark suites for contraction-order optimizers adopt per-instance certified brackets—not only optimizer-vs-optimizer deltas—as the standard of evidence.

Acknowledgments

The measurement campaign, including all attempt implementations, bounds, certificates, and reports, was executed by an autonomous AI research loop (Claude, Anthropic) under the author’s direction, with every scored result produced by the sandboxed validator described in Sec. 7. [Funding acknowledgments to be added.]

References

- [1] Sheela Orgler and Mark Blacher. Optimizing tensor contraction paths: A greedy algorithm approach with improved cost functions. *ArXiv*, abs/2405.09644, 2024.
- [2] R. Pfeifer, J. Haegeman, and F. Verstraete. Faster identification of optimal contraction sequences for tensor networks. *Physical review. E, Statistical, nonlinear, and soft matter physics*, 90 3:033315, 2013.
- [3] Johnnie Gray and S. Kourtis. Hyper-optimized tensor network contraction. *Quantum*, 5:410, 2020.
- [4] G. Kalachev, Pavel Panteleev, and M. Yung. Multi-tensor contraction for xeb verification of quantum circuits. 2021.
- [5] Optimizing tensor network partitioning using simulated annealing. 2025.
- [6] Igor L. Markov and Yaoyun Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008.
- [7] E. Dumitrescu, A. L. Fisher, T. Goodrich, T. Humble, Blair D. Sullivan, and Andrew Wright. Benchmarking treewidth as a practical component of tensor network simulations. *PLoS ONE*, 13, 2018.
- [8] Cameron Ibrahim, Danylo Lykov, Zichang He, Y. Alexeev, and Ilya Safro. Constructing optimal contraction trees for tensor network quantum circuit simulation. *2022 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–8, 2022.
- [9] Christoph Staudt, Mark Blacher, Julien Klaus, Farin Lippmann, and Joachim Giesen. Improved cut strategy for tensor network contraction orders. pages 27:1–27:19, 2024.
- [10] Yong Liu, Yaojian Chen, Chu Guo, Jiawei Song, X. Shi, Lin Gan, Wenzhao Wu, Wei Wu, H. Fu, Xin Liu, Dexun Chen, Zhifeng Zhao, Guangwen Yang, and Jiangang Gao. Verifying quantum advantage experiments with multiple amplitude tensor network contraction. *Physical review letters*, 132 3:030601, 2022.
- [11] Jin-Guo Liu and contributors. omeco: One more einsum contraction order, 2026. Rust library with Python bindings, v0.2.6.
- [12] Jin-Guo Liu and contributors. Omeinsum-contractionorders.jl, 2024. Julia package for einsum contraction-order optimization.
- [13] H. Tamaki. Heuristic computation of exact treewidth. *ArXiv*, abs/2202.07793, 2022.
- [14] H. Tamaki. Positive-instance driven dynamic programming for treewidth. *Journal of Combinatorial Optimization*, 37:1283 – 1311, 2017.
- [15] Ben Strasser. Computing tree decompositions with flowcutter: Pace 2017 submission. *ArXiv*, abs/1709.08949, 2017.